
Oliver Krauss

Further Topics in Deep Learning

AIN2IL | MTD.ba SS2026

Hagenberg 24.06.2026

HAGENBERG | LINZ | STEYR | WELS



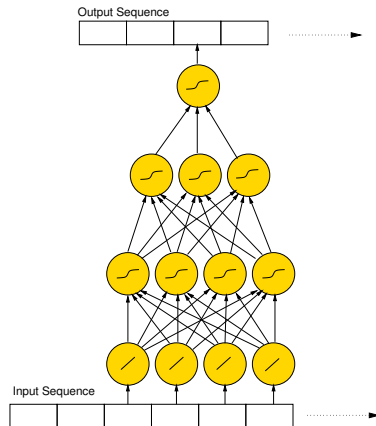
UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Time Series/Sequence Analysis With ANNs?

- Feedforward neural networks require vectorial inputs. Therefore, they cannot be applied to time series or sequences directly.
- One option is to apply them to (sliding) windows.
- The obvious disadvantage of this simple approach is that windows are treated independently and no learning across windows can take place.

Time Series/Sequence Analysis With ANNs?

- Feedforward neural networks require vectorial inputs. Therefore, they cannot be applied to time series or sequences directly.
- One option is to apply them to (sliding) windows.
- The obvious disadvantage of this simple approach is that windows are treated independently and no learning across windows can take place.

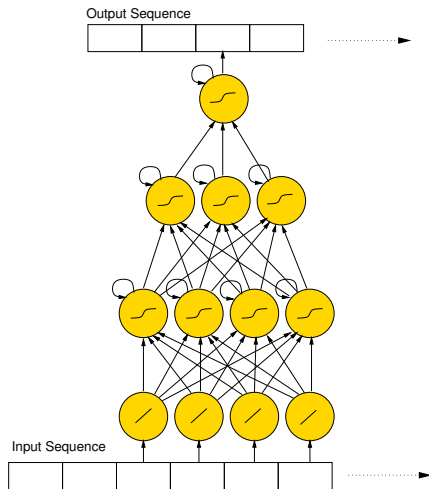


Recurrent Neural Nets (RNNs)

- *Recurrent neural networks (RNNs)* provide an alternative, where “recurrent” means that the network has connection cycles.
- There are several different RNN architectures.
- After each evaluation (for one window, in time step t), the activations are kept and potentially used as inputs in time step $t + 1$, so the generalization of the forward pass is straightforward for RNNs.
- The backpropagation algorithm can also be generalized to RNNs; this is typically called *backpropagation through time*.

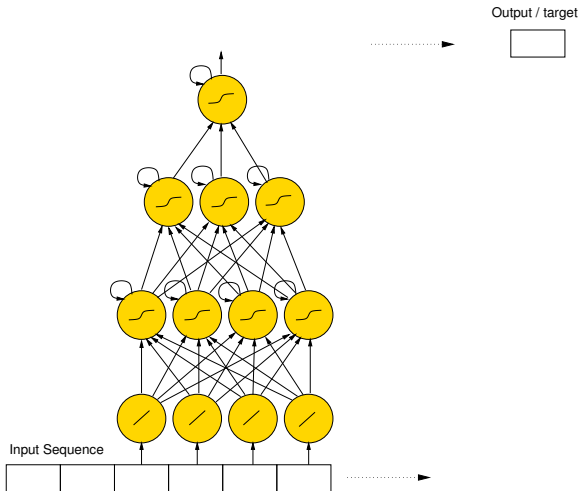
Recurrent Neural Example

Example of RNN with output sequence:



Recurrent Neural Example

Example of RNN with single output/target (output emitted only in last step)



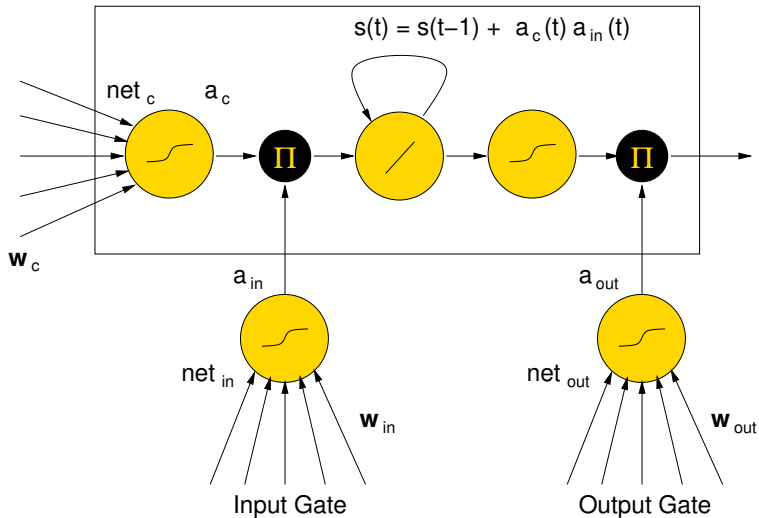
RNNs and Vanishing Gradients

- Standard RNNs with sigmoid activations are particularly prone to the vanishing gradient problem (actually, this problem has been formulated/discussed for RNNs first): errors/deltas decline (or explode) quickly when back-propagating through time.
- The consequence is that only short time lags between inputs and output signals can be learned correctly (up to about 10 time steps).

Long Short-Term Memory (LSTM)

- In order to overcome the vanishing gradient problem in RNNs, Hochreiter and Schmidhuber (1997) have introduced *Long Short-Term Memory (LSTM)* networks.
- Apart from a standard input unit, an LSTM memory cell has three main components:
 1. A *linear self-connected memory unit* (the linear activation facilitates constant error flow and thereby avoids vanishing gradients)
 2. A *multiplicative input gate* to protect the memory cell from irrelevant inputs
 3. A *multiplicative output gate* to protect the outputs (or other connected units) from currently irrelevant memory contents

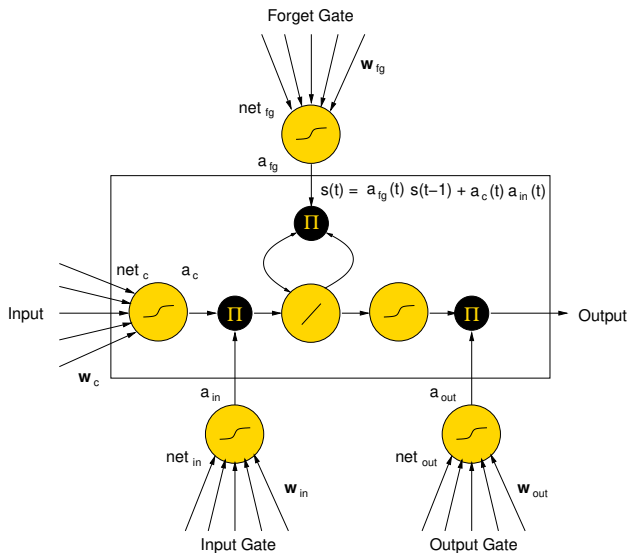
Architecture of an LSTM Memory Cell



Long Short-Term Memory (LSTM)

- LSTM memory units are trained with a so-called *truncated backpropagation* algorithm.
- Special update rules are necessary to account for the multiplicative gates.
- By avoiding the vanishing gradient problem, LSTM networks can learn across time lags of 1,000 or more time steps.
- The LSTM memory cell is just a building block. LSTM memory cells can be combined in various ways (see examples below).
- Since the introduction of LSTM, various variants have been introduced (e.g. forget gates, peepholes).
- *Gated Recurrent Units (GRUs)* follow similar ideas; nonchalantly speaking, they are simplified LSTM cells without output gate, but with a forget gate.

LSTM Memory Cell With Forget Gate



LSTM Networks I

- Like neurons in any other ANN, an LSTM cell alone is quite limited; so, LSTM cells are typically only the building blocks of large networks.
- LSTM cells are combined to one or multiple layers. The neurons inside one layer are connected via so-called recurrent connections.
- Hochreiter's original idea was to use only recurrent connections for the gates.
- Common implementations of LSTM cells connect the gates to all other LSTM cells in an LSTM layer, but also to the input.
- It is possible and common to add fully connected layers on top of LSTM layers.

LSTM Networks II

Different ways of processing are possible for LSTM networks:

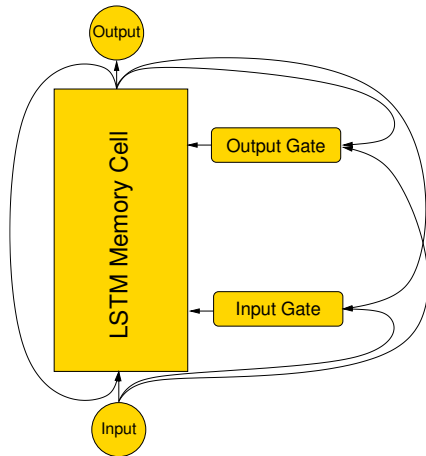
1. Emit output immediately in each time step or with a certain delay (e.g. for time series forecasting)
2. Emit output only after processing a sequence (e.g. for sequence classification)
3. Combine two LSTM networks:

Encoder: LSTM network that emits outputs after processing a sequence

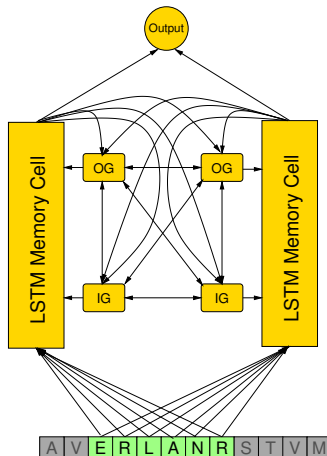
Decoder: LSTM network that takes the output of the encoder and turns it into a sequence

This model is often called *sequence to sequence learning*.

Extremely Simple LSTM Net for Time Series Analysis



LSTM Network for Time Sequence Classification



Sequence to Sequence Learning

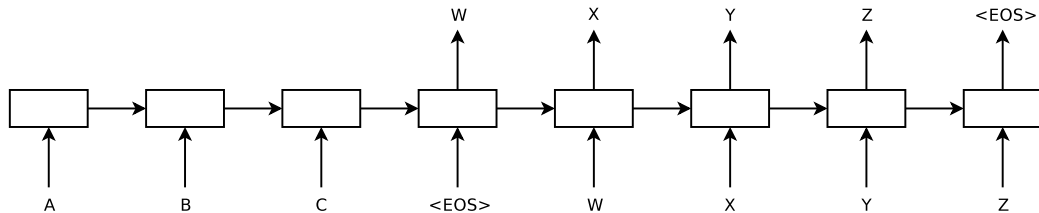


Figure: Source: I. Sutskever, O. Vinyals & Q.V. Le, NIPS 2014

Language Processing With LSTM Networks

- While there are some approaches to process language letter-wise, the by far most common approach is to process word-/token-wise.
- Since LSTM networks, like any other ANN, can only process real numbers, a mapping of words/tokens to a vectorical representation is necessary.
- For very limited vocabulary, one-hot encodings can be used.
- For more extensive vocabulary, it is common to use *word embeddings*, i.e. mapping one-hot-encoded binary vectors to a real-valued vector space such that words with similar meanings are mapped to close-by locations.
- Prior to one-hot encoding (and embedding), some pre-processing steps are also common (e.g. removal of stop words and punctuation, stemming).

Word Embeddings

Global Vectors for Word Representation (GloVe): The vector embedding is achieved by using matrix factorization on the matrix of word co-occurrences.

Word2Vec: A shallow two-layer neural network is trained that predicts sentence context, either by predicting the word in the middle from the context of surrounding words (CBOW) or by predicting surrounding words from the word in the middle (skip-gram). Then the output layer is chopped off and the mapping to the hidden layer can be used as word-to-vector embedding.

Bidirectional Encoder Representations from Transformers (BERT): A highly sophisticated encoder network that allows for taking into account in which context a word occurs.

Pre-trained embeddings/encoders are available for use in custom applications.

Word Embeddings Example

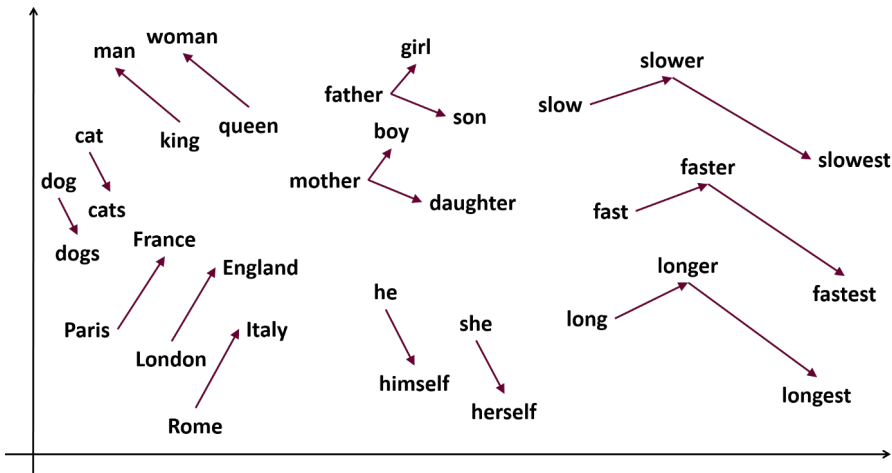


Figure: Source: S. Pal, medium.com, 2019

LSTM: The Current Hype

Some benchmark records of 2014 achieved by LSTM:

- Text-to-speech synthesis (Fan *et al.*, Microsoft, Interspeech 2014)
- Language identification (Gonzalez-Dominguez *et al.*, Google, Interspeech 2014)
- Large vocabulary speech recognition (Sak *et al.*, Google, Interspeech 2014)
- Prosody contour prediction (Fernandez *et al.*, IBM, Interspeech 2014)
- Medium vocabulary speech recognition (Geiger *et al.*, Interspeech 2014)
- English to French translation (Sutskever *et al.*, Google, NIPS 2014)
- Audio onset detection (Marchi *et al.*, ICASSP 2014)
- Social signal classification (Brueckner & Schuler, ICASSP 2014)
- Arabic handwriting recognition (Bluche *et al.*, DAS 2014)
- Image caption generation (Vinyals *et al.*, Google, 2014)
- Video to textual description (Donahue *et al.*, 2014)

LSTM in Companies

LSTM @Google:

- Neural Machine Translation System (NMT)
- Google Voice Transcription (Android speech recognizer)

LSTM @Microsoft:

- Photo-real talking head with deep bidirectional LSTM
- Spoken language understanding using LSTM
- Text-to-speech synthesis with bidirectional LSTM-based RNN

LSTM @facebook:

- Text analysis

LSTM @Apple:

- Siri

More Cool Examples

- Convolutions can be combined with recurrency in order to facilitate genuine processing of video sequences, e.g. for *video forecasting*.
- We highlighted encoder-decoder transformer networks above. This idea is in no way limited to using the same type of data on input and output side:
 - It is possible to encode the content of an image or video into an abstract representation and to decode it into language → image-to-text captioning.
 - Although this is still in its infancy, it is also possible to encode a textual description into an abstract representation and to decode it into an image using a deconvolution network → text-to-image translations.

Example: Image Captions



"man in black shirt is playing guitar."



"construction worker in orange safety vest is working on road."



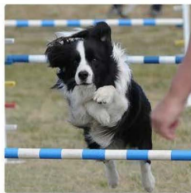
"two young girls are playing with lego toy."



"boy is doing backflip on wakeboard."



"girl in pink dress is jumping in air."



"black and white dog jumps over bar."



"young girl in pink shirt is swinging on swing."



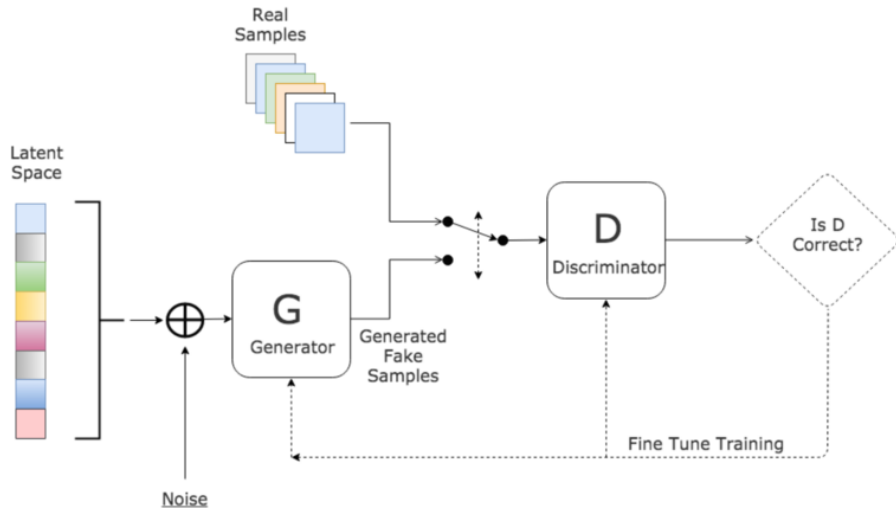
"man in blue wetsuit is surfing on wave."

Figure: Source: A. Karpathy & F.-F. Li, U. Stanford

Generative Adversarial Networks (GANs)

- Generative Adversarial Networks (GANs) have been introduced by Ian Goodfellow. They provide a deep learning approach to *generative modeling*. So, GANs try to mimick the data generation process that underlies a data set (resp. its underlying probability distribution).
- The main idea is to have two networks that sort of “play against each other”:
 1. The **discriminator** network tries to distinguish between real data and artificially generated data.
 2. The **generator** network tries to create data that the discriminator can no longer distinguish from real data.
- GANs have been called “[...] *the coolest idea in machine learning in the last twenty years*” (Yann LeCun, Chief AI Scientist at Facebook, 2016).

GAN Architecture



Discriminator Network

Goal: distinguish between real and fake images

Input format: same as real data samples

Output format: binary output (1 = real / 0 = fake); typically a single output unit with sigmoid activation;

Training:

1. Real and fake samples (see generator below) are presented to the discriminator network.
2. Optimization is done by adapting the discriminator's weights such that the binary cross entropy loss is *minimized*.

Generator Network

Goal: produce samples the discriminator cannot distinguish from real samples anymore

Input format: random vectors of certain dimensionality (so-called *latent space*)

Output format: same as real data samples

Training:

1. Random inputs are generated and the outputs of the generator are computed to produce fake samples.
2. The fake samples are presented to the discriminator along with real samples.
3. Optimization is done by adapting the generator's weights such that the activation of the discriminator (discriminator's probability that the fake sample is real) is *maximized*.

GAN Training in a Nutshell

1. Assemble a batch of real and fake samples:
 - Select some real samples
 - Produce some fake samples by propagating random inputs through the generator network
2. Propagate the batch through the discriminator network
3. Adapt weights:
 - The “error signal” for maximizing the discriminator’s probability of considering fake samples as real is propagated through the discriminator without adapting the weights and further propagated to the generator to perform its weight updates.
 - The weight updates in the discriminator are done as usual.

GAN Technical Issues

- The two networks learn simultaneously. In some way, they “bootstrap” each other. In order to ensure that both learn well, their architectures and learning strategies need to fit well. Nonchalantly speaking, one idea is to have slower adaptation in the generator than in the discriminator (cf. Heusel *et al.*, 2018).
- The generator might only create samples from certain sub-groups. This issue is called *mode collapse*. It can be counteracted with special loss functions.

GAN Applications

- The most classical applications involve images:
 1. The discriminator is a typical CNN.
 2. The generator is a so-called *deconvolution network*, i.e. a CNN where the resolution increases in each convolutional layer.
- The GAN approach is applicable to all types of data which can be processed and generated with neural networks:
 - Sequences: text, musical scores
 - Audio
 - Video

Example: StyleGAN



Figure: Source: T. Karras, S. Laine & T. Aila, arXiv, 2018

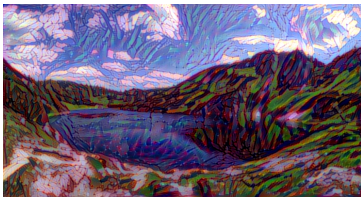
Neural Style Transfer

- Even though it is not exactly based on GANs, there is another related technology for creating data called *neural style transfer*.
- It starts from a content sample (C) and a style sample (S) . The goal is to produce a sample that has a similar content as the C, but a style similar to S. This is accomplished by two different loss functions, one for content similarity and one for style similarity.

Example: Neural Style Transfer



Example: Neural Style Transfer



Ethical issues

- GANs and related techniques have become so powerful that they can easily produce samples that not even a human can distinguish from real ones. This is subsumed under the term *deep fakes* and has gained widespread attention as a threat.
- Some areas where deep fakes have been used or at least been shown for demonstration purposes:
 - Fake news and hoaxes (e.g. speeches of politicians that have never been given)
 - Generation of fake profiles in social media and dating apps
 - Celebrity p**n, revenge p**n

Wrap-Up

- Without any doubt, deep networks are *the most powerful machine learning tools ever*. They have excelled in virtually every domain, e.g. computer vision, speech & language processing, and many more.
- Despite the practical successes, the theoretical foundations why and under which conditions deep networks work are lagging far behind.
- The spectrum of variants is hard to survey, and the choice of good parameters is both crucial and tricky.
- Learning good representations of complex data, such as, high-res images, requires excessive amounts of training data and excessive computational power (supercomputers, GPUs).
- Despite the challenges, the field is more accessible than ever before. Almost everybody can use and even train sophisticated networks. There is a host of open source software, great open solutions, and magnificent tutorials.
- SO, USE THIS WONDERFUL SET OF TOOLS! --- **DEEP LEARNING IS COOL!**